

XR Slate: Swiping and Layout Adjustment for Text Entry

Theodore Okamura*

Regis Kopper PhD[†]

Department of Computer Science
University Of North Carolina at Greensboro
Greensboro, NC, USA

ABSTRACT

In the expansive realm of virtual reality (VR) and digital interfaces, this project introduces XR Slate, an innovative text entry method, to address the challenges associated with the prevailing ray-casting approach. XR Slate mitigates ray-casting difficulty in precisely hitting specific keys due by progressively refining the virtual keyboard using intuitive swiping gestures, eliminating unwanted keys and enhancing the accessibility and user-friendliness of the text entry process.

1 INTRODUCTION

Virtual Reality (VR) and digital interfaces have ushered in a new era of immersive experiences, challenging us to re-imagine conventional methods of interaction. Amidst this technological landscape, the quest to optimize text entry mechanisms within VR environments has become a pivotal exploration. This project is at the forefront of addressing the challenges associated with the most common method for text entry in VR: ray-casting. Although widely used, ray-casting often results in imprecise inputs and user frustration, particularly because of the difficulty of hitting specific keys. In response to these challenges, this research proposes eXtended Reality Swiping and Layout Adjustment for Text Entry (XR Slate), an innovative text entry method that transcends the constraints of traditional ray-casting. XR Slate combines raycasting and progressive refinement [2], allowing the reduction and rearrangement of keys to maximize typing performance. This project offers an initial attempt at the application of a progressive refinement interaction style aimed at contributing valuable insight into the ongoing discourse surrounding the efficiency and user experience of text entry in virtual reality.

2 BACKGROUND

In the exploration of text entry methods within virtual reality (VR) and digital interfaces, the literature has outlined crucial criteria for evaluation, including accuracy, speed, learnability, usability, and physical exertion. The HGR method [3] introduces an innovative approach to text input, allowing users to draw letters with their fingers and emphasizing improvements in accuracy, learnability, and usability. However, concern over speed remains a challenge for such methods.

Exploring an alternative method, the Punchkeyboard keyboard [1] offers another novel text-entry alternative, with users interacting with virtual keys using virtual drumsticks. However, a limitation arises as the drum-like VR keyboard requires physical proximity of the keys to the user, impacting the field of view within the VR experience.

The method proposed in this study, XR Slate, relies heavily on the detection of swiping gestures. Inspired by the work of Shanthakumar et al. [4], which defined specific parameters for gesture detection using an inertial measurement unit (IMU)-based motion capture sys-

tem. The divergence in hardware between IMUs and VR controllers created the need for adaptation, ensuring broader accessibility and fostering a user-friendly text entry experience for individuals immersed in VR environments, as most users commonly utilize VR controllers rather than external motion sensors.

Finally, XR Slate uses a progressive refinement interaction style to increase the accuracy of text entries. Selection by progressive refinement [2] uses several discrete low-precision intermediate steps to reduce the set of selectable objects. The SQUAD technique, proposed by Kopper et al. [2] uses sphere-casting to select candidate objects in highly cluttered environments as the initial refinement phase, followed by an arrangement of those objects in a quad-menu with swipe-like selections of quadrants to dynamically refine the set of selectable objects. XR Slate borrows the quad-menu refinement method of SQUAD and uses swiping gestures to reduce the number and rearrange the keys in the virtual keyboard.

3 XR SLATE DESIGN

XR Slate fosters a dynamic and adaptive layout to optimize operational comfort and efficiency in virtual reality (VR) text entry. Users initiate real-time modifications to the keyboard layout through lateral swiping actions, strategically reducing the pool of available keys, while preserving their corresponding directional associations. This design allows users to concentrate on a specific subset of larger keys, thereby elevating precision, as shown in Fig. 1. The self-scaling mechanism of the keyboard ensures proportional adjustments in both size and position, thereby enhancing visibility and accessibility during interactions in VR environments.

Throughout all the refinement phases, word suggestions based on a simple dictionary were displayed to the user. The desired words can be selected by pointing the joystick in the following directions: forward left, forward, and forward right.

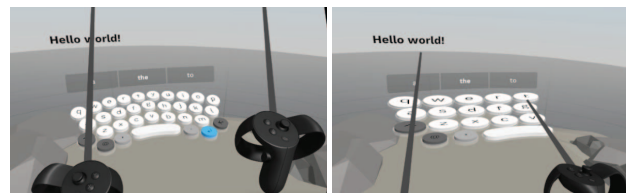


Figure 1: (left) Initial state of the virtual keyboard without any refinement. (right) Virtual keyboard after one swipe refinement, showcasing the adaptive layout.

3.1 Implementation Details

XR Slate was implemented in Unity and C# forked from the open-source Punchkeyboard project.¹ The interface supports traditional XR Slate, Punchkeyboard, and traditional raycasting.

Given that the original project was last updated in 2015, adjustments were essential to align with significant changes in the Unity and SteamVR versions. Notably, SteamVR 2.0 introduced a new API based on “actions,” necessitating modifications in the code. Adapting

¹<https://github.com/rjth/Punchkeyboard>

*tmokamura@uncg.edu

[†]rakopper@uncg.edu

to these changes involved replacing outdated methods, such as button press subscriptions, with a new action-based approach. In addition, various class and method names have been updated to enhance readability.

3.2 Gesture Detection Adaptation

The gesture detection module in `MotionDetection.cs` borrows its parameters and concepts from Shanthakumar et al. [4] to identify swipe gestures accurately. Notably, parameters such as `AngularVelocityThreshold`, `MotionDurationThreshold`, and `MotionCooldown` are crucial for determining a valid swipe. However, adjustments were made to accommodate the absence of additional data, such as the wrist direction vector, which was not captured by standard VR controllers. Instead, maintaining the `previousPosition` aids in calculating the intended direction.

3.3 Algorithm for Keyboard Refinement

The keyboard refinement process is a multi-step operation that initiates with the transformation of `refinementString`, which is a representation of the user's swipe sequence, into upper and lower bounds. These bounds play a crucial role in determining the range of keys to be filtered during refinement. Subsequently, the corresponding keys are selectively enabled or disabled, laying the foundation for subsequent adjustments.

The reorganization phase involves dynamic calculations of spacing, scaling, and local positions to ensure adaptability to the evolving configuration. Notably, this process considers the unique spatial requirements of a large space key. Such meticulous attention guarantees a refined and responsive keyboard layout that optimizes the user's text entry experience within virtual reality environments.

3.4 Interpolation Method

To ensure a smooth user experience, interpolation was applied for position and scale updates on the keyboard. Maintaining `targetPositions` and `targetScales` maps allows for efficient destination value storage. During each frame, `Unity Vector3.Lerp` calculated the necessary value changes to avoid abrupt transitions.

4 ENHANCEMENTS AND FUTURE DIRECTIONS

Following a pilot study, several limitations and usability issues emerged, prompting considerations for improvements in the XR late method. While some changes are straightforward, others may involve substantial modifications of the overall interaction technique.

4.1 Swapped Swiping Directions

We should consider swapping the swiping directions to enhance user intuitiveness. Instead of preserving the section swiped toward, users may find it more natural to remove the targeted section, as in a sweeping motion. This adjustment, though relatively simple to implement, has the potential to enhance overall usability.

4.2 Advanced Predictive Word Suggestions

To address the current limitations of word suggestions, there is a need for a more advanced algorithm. An improved algorithm would significantly enhance the usability of all three text-entry techniques implemented in this project, providing users with more accurate and context-aware word predictions.

4.3 Visual Feedback for Active Key

Integrating visual feedback by highlighting the key that is active for selection could offer users a clearer indication of their intended input. This enhancement aims to improve accuracy, reduce frustration, and provide real-time visual cues. Implementing this feature in both XR Slate and the regular ray-casting technique would contribute to a better understanding of the system's response to user interactions.

4.4 Fish Eye Effect for Key Interaction

Exploring the implementation of the fish-eye effect as a text-entry method would introduce a novel interaction model. With this ap-

proach, pointing at a key can magnify it or bring it into focus, thereby offering a visually engaging input experience. This method may be preferred when more natural and dynamic interactions are required.

4.5 Reactive Method: Wait and Press

Another text-entry technique, in which users wait until the desired key is hovered and then press a button, introduces an intriguing approach. Preliminary findings suggest a slower input speed,² but its increased accessibility and reduced physical exertion warrant further exploration and dedicated user studies to assess its effectiveness.

5 PROPOSED USER STUDY

To ensure the effectiveness of XR Slate, it is imperative to conduct a comparative evaluation. We propose that XR Slate should be compared with Raycasting and Punchkeyboard for text input. Raycasting is the most common text entry technique, and the Punchkeyboard is an improvement. It is important to collect a variety of measures, including *text input speed*, *text input accuracy*, *usability*, *user comfort*, *fatigue*, *user satisfaction*, *preference*, and *general feedback*. In outlining the procedure for an envisioned study, participants would undergo VR device familiarization, engage in inputting predefined strings, and subsequently participate in surveys. These surveys should draw on established metrics, such as the System Usability Index and the NASA Task Load Index, ensuring a standardized and proven approach to measure the desired outcomes comprehensively.

6 CONCLUSION

XR Slate offers a potentially effective solution to common challenges in the realm of VR text entry. By integrating swiping gestures with ray-casting, the project aims to improve accessibility and user friendliness. XR Slate's flexibility prioritizes user comfort and efficiency using a self-scaling feature.

Technical aspects, including code modernization and gesture detection, were approached carefully. The user-centric keyboard refinement algorithm responds to swipe sequences, thereby reflecting a thoughtful design philosophy.

A potential user study would capture essential measures, guiding XR Slate's continuous improvement and overall usability. In its modest pursuit, XR Slate aims to contribute insights into broader conversations to enhance VR text entry experiences.

In conclusion, XR Slate offers an alternative text input technique that may advance the exploration of VR interfaces. Its commitment to simplicity and a genuine desire to enhance VR interactions reflect a humble contribution to the evolving field of immersive experiences.

The complete codebase for this project is available on GitHub.³

ACKNOWLEDGMENTS

Thanks are extended to the contributors of the original Punchkeyboard project, which served as the foundation for this project.

REFERENCES

- [1] C. Boletsis and S. Kongsvik. Text input in virtual reality: A preliminary evaluation of the drum-like VR keyboard. *Technologies*, 7(2):31, 2019.
- [2] R. Kopper, F. Bacim, and D. A. Bowman. Rapid and accurate 3D selection by progressive refinement. In *Proc. IEEE 3DUI*, 2011.
- [3] N. Nooruddin, R. Dembani, and N. Maitlo. HGR: Hand-Gesture-Recognition based text input method for AR/VR wearable devices. *Proc. IEEE SMC*, 2020.
- [4] V. A. Shanthakumar, C. Peng, J. Hansberger, L. Cao, S. Meacham, and V. Blakely. Design and Evaluation of a Hand Gesture Recognition Approach for Real-Time Interactions. *Multimed. Tools Appl.*, 79(25-26), 2020.

²<https://github.com/AttackOfTheMoons/reflex-evaluation>

³<https://github.com/AttackOfTheMoons/XRSlate>